Министерство образования Республики Беларусь

Учреждение образования «Брестский государственный технический университет»

Кафедра ИИТ

Лабораторная работа №1

По дисциплине «Криптографические методы защиты информации»

Тема: «Бинарная классификация»

Выполнил:

Студент 3 курса

Группы ИИ-26 (2)

Суховило В.С.

Проверила:

Андренко К.В.

Брест 2026

Цель работы: Изучить принципы бинарной классификации и реализовать однослойную нейронную сеть (персептрон) для решения задачи классификации с использованием **пороговой** функции активации, а также исследовать процесс обучения модели с применением среднеквадратичной ошибки (MSE).

Задачи лабораторной работы:
1. Реализовать алгоритм обучения однослойной нейронной сети с использованием **MSE** в качестве функции ошибки **(дельта-правило)**.
2. Провести обучение сети с **разными значениями шага обучения** и построить **график зависимости** MSE от номера эпохи.
3. Выполнить визуализацию результатов классификации:
   - исходные точки обучающей выборки,
   - разделяющую линию (границу между двумя классами).
4. Реализовать режим функционирования сети:
   - пользователь задаёт произвольный входной вектор,
   - сеть вычисляет выходной класс,
   - соответствующая точка отображается на графике,
   - для корректной визуализации рекомендуется выбирать значения из диапазона ВСТАВИТЬ СВОЙ ДИАПАЗОН, например $-0.5 \leq x1, x2 \leq 1.5$

5. Написать вывод по выполненной работе.
Допускается применение **математических** и **графических** библиотек
ML-библиотеки и **ML-фреймворки** использовать **нельзя** (например: scikit-learn, TensorFlow, PyTorch - запрещены)

Вариант 5:

5.

x_1	x_2	e
2	6	0
-2	6	1
2	-6	0
-2	-6	0

Код программы:
```
import numpy as np
import matplotlib
matplotlib.rcParams['toolbar'] = 'None'
import matplotlib.pyplot as plt

X = np.array([
   [2, 6],
   [-2, 6],
   [2, -6],
```

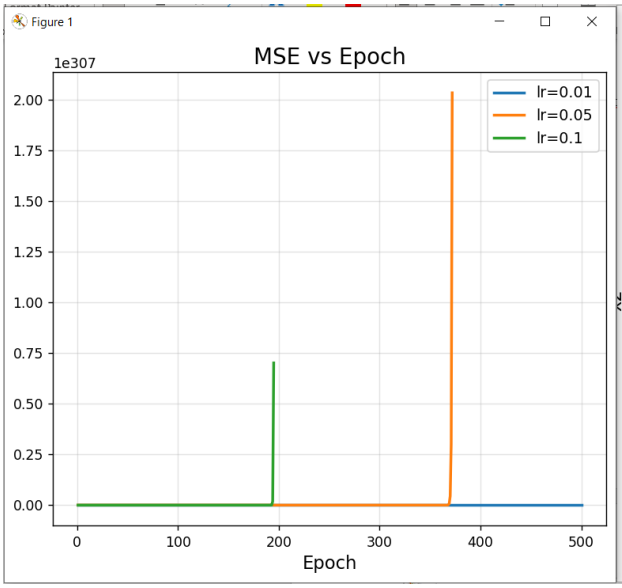
```
    [-2, -6]
], dtype=float)

E = np.array([0, 1, 0, 0], dtype=int)
T = np.where(E == 0, -1, 1).astype(float)

X1_MIN, X1_MAX = -3.0, 3.0
X2_MIN, X2_MAX = -7.0, 7.0

def step(u):
    return 1 if u >= 0 else 0

def forward_linear(X, w, b):
    return X @ w + b

def forward_class(X, w, b):
    u = forward_linear(X, w, b)
    return np.array([step(val) for val in u], dtype=int)

def train_mse_lms(X, T, lr=0.05, epochs=500, w0=None, b0=0.0):
    n_features = X.shape[1]
    w = np.zeros(n_features, dtype=float) if w0 is None else np.array(w0, dtype=float).copy()
    b = float(b0)
    mse_history = []

    for _ in range(epochs):
        y_lin = forward_linear(X, w, b)
        err = T - y_lin
        mse = np.mean(err ** 2)
        mse_history.append(mse)

        grad_w = -(2.0 / X.shape[0]) * (X.T @ err)
        grad_b = -(2.0 / X.shape[0]) * np.sum(err)

        w = w - lr * grad_w
        b = b - lr * grad_b

    return w, b, np.array(mse_history)

def plot_mse(histories, lrs):
    plt.figure(figsize=(10, 6))
    for lr, h in zip(lrs, histories):
        plt.plot(np.arange(1, len(h) + 1), h, linewidth=2, label=f"lr={lr}")
    plt.title("MSE vs Epoch", fontsize=16)
    plt.xlabel("Epoch", fontsize=13)
    plt.ylabel("MSE", fontsize=13)
    plt.grid(True, alpha=0.35)
    plt.legend(fontsize=11)
    plt.tight_layout()
```

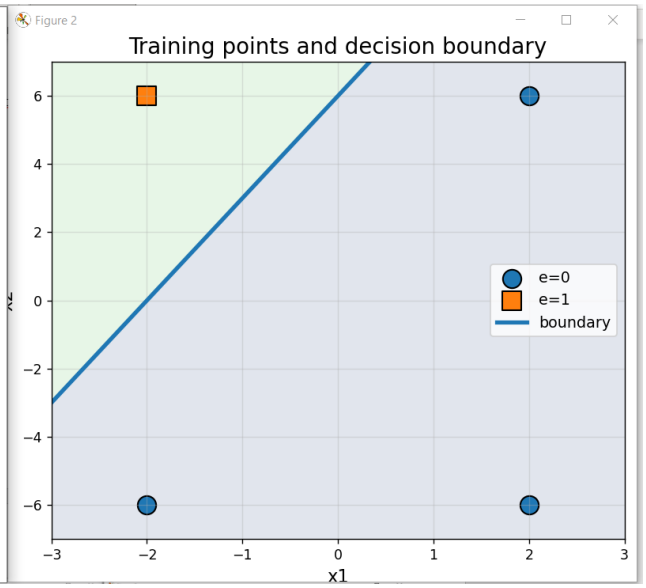
```
def plot_data_and_boundary(X, E, w, b):
    plt.figure(figsize=(10, 8))
    plt.xlabel("x1", fontsize=13)
    plt.ylabel("x2", fontsize=13)
    plt.title("Training points and decision boundary", fontsize=16)
    plt.grid(True, alpha=0.35)

    xx, yy = np.meshgrid(
        np.linspace(X1_MIN, X1_MAX, 300),
        np.linspace(X2_MIN, X2_MAX, 300)
    )
    zz = w[0] * xx + w[1] * yy + b
    plt.contourf(xx, yy, zz, levels=[-1000, 0, 1000], alpha=0.15)

    X0 = X[E == 0]
    X1c = X[E == 1]

    if len(X0) > 0:
        plt.scatter(X0[:, 0], X0[:, 1], marker="o", s=180, edgecolors="black", linewidths=1.2, label="e=0")
    if len(X1c) > 0:
        plt.scatter(X1c[:, 0], X1c[:, 1], marker="s", s=180, edgecolors="black", linewidths=1.2, label="e=1")

    w1, w2 = w[0], w[1]

    if abs(w2) >= 1e-12:
        x1_vals = np.linspace(X1_MIN, X1_MAX, 300)
        x2_vals = -(w1 * x1_vals + b) / w2
        plt.plot(x1_vals, x2_vals, linewidth=3, label="boundary")
    elif abs(w1) >= 1e-12:
        x1_const = -b / w1
        plt.axvline(x=x1_const, linewidth=3, label="boundary")

    plt.xlim(X1_MIN, X1_MAX)
    plt.ylim(X2_MIN, X2_MAX)
    plt.legend(fontsize=11)
    plt.tight_layout()

def interactive_mode(w, b):
    plt.ion()
    fig, ax = plt.subplots(figsize=(10, 8))
    ax.set_xlabel("x1", fontsize=13)
    ax.set_ylabel("x2", fontsize=13)
    ax.set_title("Interactive mode", fontsize=16)
    ax.grid(True, alpha=0.35)
    ax.set_xlim(X1_MIN, X1_MAX)
    ax.set_ylim(X2_MIN, X2_MAX)

    xx, yy = np.meshgrid(
```

```
    np.linspace(X1_MIN, X1_MAX, 300),
    np.linspace(X2_MIN, X2_MAX, 300)
)
zz = w[0] * xx + w[1] * yy + b
ax.contourf(xx, yy, zz, levels=[-1000, 0, 1000], alpha=0.15)

X0 = X[E == 0]
X1c = X[E == 1]

if len(X0) > 0:
    ax.scatter(X0[:, 0], X0[:, 1], marker="o", s=180, edgecolors="black", linewidths=1.2, label="train e=0")
if len(X1c) > 0:
    ax.scatter(X1c[:, 0], X1c[:, 1], marker="s", s=180, edgecolors="black", linewidths=1.2, label="train e=1")

w1, w2 = w[0], w[1]

if abs(w2) >= 1e-12:
    x1_vals = np.linspace(X1_MIN, X1_MAX, 300)
    x2_vals = -(w1 * x1_vals + b) / w2
    ax.plot(x1_vals, x2_vals, linewidth=3, label="boundary")
elif abs(w1) >= 1e-12:
    x1_const = -b / w1
    ax.axvline(x=x1_const, linewidth=3, label="boundary")

ax.legend(fontsize=11)
fig.tight_layout()
fig.canvas.draw()
fig.canvas.flush_events()

while True:
    s = input("x1 x2 (or q): ").strip()
    if s.lower() in ("q", "quit", "exit"):
        break
    try:
        parts = s.replace(",", " ").split()
        if len(parts) != 2:
            print("Enter 2 numbers")
            continue
        x1v = float(parts[0])
        x2v = float(parts[1])
    except:
        print("Input error")
        continue

    u = x1v * w[0] + x2v * w[1] + b
    cls = step(u)

    marker = "x" if cls == 0 else "*"
    ax.scatter([x1v], [x2v], marker=marker, s=220, linewidths=2.2)
```

```
        fig.canvas.draw()
        fig.canvas.flush_events()

        print(f"class = {cls}")

    plt.ioff()
    plt.show()

def main():
    lrs = [0.01, 0.05, 0.1]
    epochs = 500

    histories = []
    best_w, best_b = None, None
    best_acc = -1

    w_init = np.array([0.0, 0.0])
    b_init = 0.0

    for lr in lrs:
        w, b, mse_hist = train_mse_lms(X, T, lr=lr, epochs=epochs, w0=w_init, b0=b_init)
        histories.append(mse_hist)

        y_pred = forward_class(X, w, b)
        acc = np.mean(y_pred == E)

        print(f"lr={lr}: w={w}, b={b:.4f}, acc={acc:.2f}")

        if acc > best_acc:
            best_acc = acc
            best_w, best_b = w.copy(), b

    plot_mse(histories, lrs)
    plot_data_and_boundary(X, E, best_w, best_b)
    plt.show()

    interactive_mode(best_w, best_b)

if __name__ == "__main__":
    main()
```

Исправленный результат :





Вывод:

изучил принципы бинарной классификации и реализовала однослойную нейронную сеть (персептрон) для решения задачи классификации с использованием пороговой функции активации, а также исследовала процесс обучения модели с применением среднеквадратичной ошибки (MSE).